Ders 17 Genişletilmiş Notu: Uygulama 2: Shell Programlama, Pipe, tr, awk, Koşullar, Döngüler ve case

Ders 17 Genişletilmiş Notu: Uygulama 2: Shell Programlama, Pipe, tr, awk, Koşullar, Döngüler ve case

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri ve Önceki Haftanın Tekrarı
- 2. Kullanıcı ve Grup Bilgilerinin Tutulduğu Dosyalar
- 3. `mv` ile Yeniden Adlandırma ve `sort`
- 4. Boru (Pipe) ile Komutları Birleştirmek
- 5. `tr` ile Karakter Dönüştürme ve Silme
- 6. `awk` ile Sütun Bazlı Filtreleme
- 7. `bc` Hesap Makinesi
- 8. Koşul Yapıları: `if`, `then`, `else`, `fi`
- 9. Döngüler: `for`, `while`, `until` ve `let`
- 10. `case` Yapısı
- 11. Sorgu Betiğinin Hata Ayıklaması
- 12. Uygulamanın Genel Değerlendirmesi ve Sonraki Uygulamalar
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Yeri ve Önceki Haftanın Tekrarı

Bu kayıt, kabuk (shell) programlama uygulamasının ikinci ve son oturumudur. Uygulamayı, ders 15'teki ilk oturumu da anlatan öğretim elemanı yürüttü. Kayıt Zoom üzerindendir ve asıl anlatım yaklaşık 00:13'te başlar. Öncesi katılımcıların toplanmasını beklemekle geçti.

Anlatıcı, geçen hafta bazı slaytlarda düzeltme gerektiğini fark ettiğini söyledi. Bu derste hem bu düzeltmelerin üzerinden geçileceğini hem de kalan konuların tamamlanacağını belirtti.

Önce geçen haftanın kısa bir tekrarı yapıldı:

- Farklı kabuklardan söz edilmişti. Kabuk, çekirdeğin (kernel) üzerinde çalışan ve bazı kolaylıklar sağlayan bir programlama dilidir.
- Klasör ve dosya işlemleri gösterilmişti: klasör ve dosya oluşturma, taşıma, silme, dosyaları terminalde görüntüleme ve düzenleme, metin düzenleyiciyi açma.
- Kullanıcı oluşturma gösterilmişti. Ancak oluşturulan kullanıcıların nerede listelendiği hatırlanamamıştı. Bu ders bu eksikle başladı (Bölüm 2).
- Dosyalara `chmod` (change mode) ile okuma, yazma ve çalıştırma hakları vermek. Anlatıcıya göre bu önemlidir, çünkü yeni oluşturulan bir dosya genelde gereken yetkilerle gelmez. Haklar kullanıcı, grup ve diğerleri için ayrı ayrı tanımlanabilir.
- Dosya tipleri: `ls -l` çıktısında her satırın ilk karakteri dosyanın tipini, sonraki karakterler ise yetkileri gösterir.

- `echo` ile bir değişkenin değerini yazdırma. Bazı kaçış dizileri (örneğin `\c`) kabuğun sürümüne göre çalışmayabilir. Bu durumda `echo -e` ile kaçış dizileri etkin hâle getirilir.
- Sistem değişkenleri büyük harfle, kullanıcı değişkenleri küçük harfle yazılır.
- Aritmetik işlemlerde `*` işareti kabukta özel anlamlı olduğu için kaçış karakteriyle (`\*`) yazılır. İfadeler ters tırnak (backtick) içinde `expr` ile de hesaplanabilir.
- Kullanıcıdan veri almak için `read` komutu kullanılır. Anlatıcı bunu C'deki `scanf`'in karşılığı olarak tanıttı. Her şeyi dosyalar üzerinden yürütemeyiz, kullanıcıyla etkileşim gerekir.
- Joker karakterler (wildcard): `*` sıfır ya da daha fazla karakterin yerini tutar. `A*` ile A'yla başlayan her ad eşleşir: "A", "AB", "ABC" gibi. `?` tam olarak tek bir karakterin yerini tutar.
- Dosyaya yazma: `>` dosyayı baştan yazar (write). `>>` var olan metni koruyarak sonuna ekler (append). Anlatıcı en önemli farkın bu olduğunu söyledi.

2. Kullanıcı ve Grup Bilgilerinin Tutulduğu Dosyalar

Geçen hafta hatırlanamayan yer, `/etc` dizinindeki dosyalardır. Anlatıcı `/etc/shadow` dosyasını açtı. Geçen hafta oluşturulan test kullanıcısı ve o gün oluşturulan ikinci test kullanıcısı bu dosyada görünüyordu.

Dosya, alanları `:` (iki nokta) ile ayrılmış satırlardan oluşur. İki nokta burada ayırıcı (separator) görevi görür:

- Birinci alan kullanıcı adıdır.
- İkinci alan şifredir. Burada şifre açık hâliyle değil, özeti (hash) alınmış hâliyle durur.
- Sonraki alanlar kullanıcının bağlı olduğu gruplar, yetkiler ve hesabın bir servis hesabı olup olmadığı gibi bilgilerle devam eder.

Anlatıcı grupların da aynı dosyada bulunduğunu söyledi ve örnek olarak sistem gruplarını gösterdi.

> **Ders sonrası not:** Derste `/etc/shadow` "kullanıcıların bulunduğu dosya" olarak tanıtıldı ve grupların da aynı dosyada olduğu söylendi. Doğrusu: kullanıcı listesi `/etc/passwd` dosyasındadır. `/etc/shadow` şifre özetlerini ve şifre süre bilgilerini tutar, yalnızca root okuyabilir. Gruplar `/etc/group` dosyasındadır. Üç dosyada da alanlar `:` ile ayrılır.

3. `mv` ile Yeniden Adlandırma ve `sort`

Anlatıcı geçen hafta hata veren dosya işlemleri örneğine geri döndü. O örnekte hata mesajı alınmıştı, ama bu hata zaten beklenen bir hataymış.

Örnekte bir dosya oluşturuluyor, sonra `mv deneme deneme.cs` benzeri bir komut çalışıyordu. Bu komut `deneme` adlı dosyanın adını `deneme.cs` olarak değiştirir. Kabukta ayrı bir yeniden adlandırma komutu kullanılmaz. Dosya taşıma komutu `mv` hem taşıma hem yeniden adlandırma için kullanılır:

- Birinci parametre taşınacak dosyadır.
- İkinci parametrede hem hedef yol hem de yeni ad verilebilir.

Aynı örnekte `sort` komutu da vardı. Anlatıcıya göre normal bir programlama dilinde sıralama için ayrı bir fonksiyon yazmak ya da eklemek gerekirken, kabukta `sort` ile satırlar çok basit biçimde sıralanabilir.

Bu arada anlatıcı, `~` (tilde) işaretinin kullanıcının ev (home) dizinine giden bir kısayol olduğunu hatırlattı.

4. Boru (Pipe) ile Komutları Birleştirmek

Geçen hafta pipe konusuna gelinmiş ve orada kalınmıştı. Anlatıcı önceki yıllarda bunu tahtaya çizerek anlattığını, şimdi sunum ekranından anlatmanın biraz zor olacağını söyledi.

Boru işareti `|`, solundaki komutu (ya da komut grubunu) çalıştırır ve onun çıktısını sağdaki komuta, yani filtreye, girdi olarak verir.

4.1 Uzun Bir Liste: `ps aux`

Anlatıcı önce `ps aux` komutunu çalıştırdı. Bu komut sistemde yaşayan bütün süreçlerin (process) durumunu gösterir. Anlatıcı bunu görev yöneticisinin (task manager) terminaldeki karşılığı olarak tanıttı:

- Bazı süreçler root kullanıcısı, bazıları ise oturumu açan kullanıcı tarafından çalıştırılmıştır.
- Her satırda hangi yoldaki hangi çalıştırılabilir dosyanın çalıştığı ve çalışma süreleri gibi bilgiler görünür.
- Komut tekrar çalıştırıldığında listenin en son hâlini verir.

Bu liste çok uzun olabilir. Önceki hafta görülen `set` komutu da benzer biçimde uzun bir liste verir: sistem değişkenleri ve kullanıcı değişkenleri. Böyle bir listenin içinde bir şey ararken kaybolabilirsiniz. Bu yüzden çıktıyı filtrelemek gerekir. Pipe burada devreye girer.

4.2 `grep` ile Filtreleme

Yalnızca root kullanıcısına ait süreçleri görmek istediğimizi düşünelim:

```
ps aux | grep -i root
```

`|` işaretinden sonra ikinci komut, yani filtreleyecek komut yazılır. Burada `grep` bir filtreleme komutudur. `-i` seçeneği büyük-küçük harf duyarlılığını kaldırır (case insensitive). Anlatıcı klavye düzenine göre `|` karakterinin farklı tuş kombinasyonlarıyla yazıldığını da belirtti.

Anlatıcı bu sırada arama kelimesini tek bir karaktere indirince aramanın beklediği gibi çalışmadığını, yanına * koyunca bütün satırları getirdiğini gösterdi.

İkinci örnek `set` çıktısı üzerindeydi:

```
set | grep USER
```

`USER` bir sistem değişkenidir. Çıktıda `USER` ve `USERNAME` gibi bu kelimenin geçtiği değişkenler, değerleriyle birlikte göründü.

Anlatıcının özeti: pipe, bir komutun çıktısını başka bir komuta aktaran yardımcı bir işaretçidir.

> **Ders sonrası not:** Derste `grep` için “yer değiştirme (replacement) yapan bir fonksiyon aynı zamanda” dendi. Bu doğru değil. `grep` yalnızca arar ve eşleşen satırları yazar. Metinde yer değiştirme için `sed` (ya da karakter düzeyinde `tr`) kullanılır.

5. `tr` ile Karakter Dönüştürme ve Silme

Bazen çıktıdaki karakterleri dönüştürmek gerekir. Bunun için `tr` (translate) komutu vardır. `tr` de pipe ile birlikte kullanılır ve bir dosyanın satırlarında istenen değişikliği yapar.

Slayttaki (10. slayt) örnekte üç kullanım vardı:

- **Aralık dönüşümü:** Bir karakter aralığı verilir. `[a-z]` aralığındaki küçük harfler `[A-Z]` aralığındaki büyük harflere dönüştürülür.
- **Tek karakter dönüşümü:** Bir aralık yerine tek bir karakter de değiştirilebilir. Karakter tek tırnak içinde yazılır.
- **Silme:** `-d` (delete) seçeneğiyle belirtilen karakterler silinir.

```
echo "merhaba;dunya" | tr '[a-z]' '[A-Z]'   # MERHABA;DUNYA
echo "a;b;c" > dosya.txt
cat dosya.txt | tr ';' ','                  # a,b,c
cat dosya.txt | tr -d ';'                   # abc
```

Anlatıcı örneği şöyle açıkladı:

- İlk satırda `echo`’nun çıktısı pipe ile `tr`’ye aktarıldı. `tr` gelen girdideki küçük harfleri büyük harfe çevirdi.
- İkinci satırda `echo` çıktısı `>` ile `dosya.txt` adlı bir dosyaya yazıldı. Dosya yoksa bu işlem dosyayı oluşturur, varsa üzerine yazar.

- Sonra dosya `cat` ile okundu, sonuç pipe ile `tr`'ye verildi ve noktalı virgüller virgülle değiştirildi.
- Son satırda `-d` seçeneğiyle noktalı virgüller tamamen silindi.

Anlatıcı pipe ve `tr` için soru olup olmadığını sordu, soru gelmedi. Bunların karmaşık şeyler olmadığını, pipe'ın bundan sonra da sık kullanılacağını söyledi.

6. `awk` ile Sütun Bazlı Filtreleme

6.1 Tablo Biçimli Dosyalar

Dosyalarımız CSV (comma separated values) gibi biçimli dosyalar olabilir. Önceki örnekteki `a;b;c` de aslında böyle bir yapıdır: noktalı virgülle ayrılan her parça bir sütuna (column) karşılık gelir. Excel'den dışa aktarılan (export) CSV dosyaları da bu biçimdedir. Ayırıcı virgül ya da noktalı virgül olabilir, bunu siz belirlersiniz.

Böyle bir dosya bir tabloyu ifade eder. Kabuk programlarında tablolar üzerinde işlem yapmak için `awk` komutu kullanılır.

6.2 Slayttaki Örnek

Örnekte önce bir dosyaya `echo` ile `isim;4;1` biçiminde bir satır yazılır. Sonra `>>` ile yeni satırlar eklenir. Ekleme bittikten sonra dosya okunur ve `awk` ile filtrelenir:

```
echo "ahmet;4;1" >  birdosya.txt
echo "ahmet;3;2" >> birdosya.txt
echo "ahmet;2;3" >> birdosya.txt
echo "ahmet;1;4" >> birdosya.txt
echo "ahmet;0;5" >> birdosya.txt
cat birdosya.txt | awk -F ';' 'int($2)>3'    # ahmet;4;1
cat birdosya.txt | awk -F ';' 'int($3)<=4'   # ilk dört satır
```

`awk -F ';'` her satırı noktalı virgüle göre ayrıştırır (parse). `-F` alan ayırıcısını (field separator) belirler. Ayrıştırılan parçalar geçici, numaralı değişkenlere atanır. Bunların içeriğine, kullanıcı değişkenlerinde olduğu gibi `$` işaretiyle erişilir:

- İlk satırda isim `$1`'e, 4 `$2`'ye, 1 `$3`'e düşer.
- İkinci satırda `$2`'nin değeri 3'tür.

Birinci `awk` komutu ikinci sütunun 3'ten büyük olup olmadığına bakar. İkinci komut üçüncü sütunu 4 ile karşılaştırır.

6.3 Örneğin Çalıştırılması

Anlatıcı örneği çalıştırdı. İki komutun çıktıları ekranda birbirine karışınca betiği satır satır ayırdı ve her seferinde yalnızca bir `awk` komutunu çalıştırdı.

- `int($2)>3` ile yalnızca `ahmet;4;1` satırı yazıldı. Çünkü ikinci sütunu 3'ten büyük olan tek satır budur.
- Sonra komut üçüncü sütuna bakacak biçimde değiştirildi ve "küçük eşit" karşılaştırmasıyla çalıştırıldı.

Ardından anlatıcı ayırıcının önemini gösterdi. `-F ';'` yerine `-F ','` yazılırsa dosyada virgül olmadığı için satır bölünemez. Bu durumda filtre işini yapamaz ve "her şeyi yazar".

> **Ders sonrası not:** Bash'te denendiğinde ayırıcı virgül yapılırsa `$2` ve `$3` boş kalır, `int()` bunları 0 kabul eder. Bu yüzden `int($3)<=4` bütün satırları yazar ($0 \leq 4$ her satırda doğru). `int($2)>3` ise hiçbir satır yazmaz. Yani "her şeyi yazar" sonucu yalnız küçük-eşit karşılaştırması için geçerlidir; derste değiştirilen komut da buydu.

Anlatıcıya göre dosyaları bir ayırıcı tanımlayarak ayrıştırmak ve sütun bazında (column-wise) filtrelemek, çalışmalarınızda büyük kolaylık sağlayabilir.

> **Ders sonrası not:** Awk slaytının açıklama satırında "tr küçük harfleri büyük harfe çevirecektir" ifadesi yer alıyordu (kısa nota göre). Bu satır önceki slayttan kalmadır,

awk örneğiyle ilgisi yoktur. Ayrıca slaytta beş `echo` komutu tek satırda ve aralarında `;` olmadan yazılmıştı. Bu hâliyle çalışmaz; komutlar yukarıdaki gibi ayrı satırlara yazılmalı ya da `;` ile ayrılmalıdır.

7. `bc` Hesap Makinesi

Kabuk programlamada hesap makinesi (calculator) olmazsa olmazdır. `bc` ile normal bir bilimsel hesap makinesinde yapılan işlemlerin çoğu yapılabilir, örneğin logaritma almak. Anlatıcı bunun önceki haftanın aritmetik işlemlerinden farklı olduğunu belirtti.

Terminalde `bc` yazıldığında ayrı bir komut satırı açılır ve işlemler orada sonucuyla birlikte gösterilir. Anlatıcı slayttaki örnekleri bu satırda yaptı:

- İlk üç ifade koşul ifadesidir (condition statement). İfade doğruysa `1`, yanlışsa `0` döner.
- Diğerleri aritmetik işlemlerdir. Örneğin `1+15` için `16`, `3-1` için `2`.

```
1+15      -> 16
3-1       -> 2
```

Anlatıcının vurgusu şuydu: terminalde yeni bir istem (prompt) açıp yazabildiğiniz her şeyi kabuk betiğine (shell script) de koyabilirsiniz. `bc` bunun en güzel örneklerinden biridir.

> **Ek bilgi (derste anlatılmadı):** `bc` yalnızca etkileşimli değil, pipe ile de kullanılabilir. Örneğin `echo "1+15" | bc` doğrudan `16` yazar. Logaritma gibi matematik fonksiyonları için `bc -l` seçeneği gerekir; bu seçenek matematik kütüphanesini yükler.

8. Koşul Yapıları: `if`, `then`, `else`, `fi`

8.1 Koşul İfadeleri

`bc`'de koşul ifadelerinin nasıl çalıştığı görülmüştü. Ama kabukta kendi koşul ifadelerinizi de yazabilirsiniz. Yalnızca "1, 3'ten küçük mü" gibi sabitleri değil, değişkenleri ve değişkenlerin içindeki değerleri de karşılaştırabilirsiniz. Karşılaştırma stringlerle de yapılabilir. Yalnızca büyüktür ve küçüktür değil, büyük eşit gibi işleçler de vardır. Yani normal programlama dillerinde gördüğümüz bütün karşılaştırmalar burada da yapılabilir.

Koşul ifadeleri köşeli parantez içinde yazılır. Karşılaştırma işleci başına `-` (tire) konarak verilir:

- Sayılar için: `-gt` (greater than, büyük), `-lt` (less than, küçük), `-eq` (equal, eşit).
- Dosyalar için: `-f` (dosya), `-d` (klasör), `-r` (okunabilir, readable), `-w` (yazılabilir, writable).

  Ders sonrası not: Altyazıda yazılabilirlik için "çizgi V" geçiyor. Doğrusu `-w`'dir (writable).

8.2 Blok Yapısı

Anlatıcı sorgu betiğini editörde açtı ve yapıyı açıkladı. `if` ile başlanır, ardından bir komut ya da koşul gelir. Bu komutun sonucu doğru ya da yanlış olarak değerlendirilir. `then` ile koşul sağlandığında yapılacak işlem, `else` ile sağlanmadığında yapılacak işlem yazılır. Blok `fi` ile kapatılır.

```
if [ koşul ]; then
  komutlar
else
  komutlar
fi
```

Diğer dillerde kod blokları ya girintiyle ya da süslü parantezle belirlenir. Kabukta ise blok, anahtar kelimeyle açılıp tersiyle kapatılır: `if` ile açılan blok `fi` ile kapanır. Benzer yapı döngülerde (`do ... done`) ve `case`'te (`case ... esac`) de vardır.

8.3 Sorgu Betiğinin İçeriği

Slayttaki sorgu betiği (`13_queries.sh`) betiğe verilen argümanları kullanarak sırayla şunları yapar:

- Bir dosya oluşturur ve içine örnek bir metin koyar. Dosya önceden varsa üzerine yazar.
- `if cat $1` ile birinci argümandaki dosyayı okumayı dener. Okuyabiliyorsa “dosya mevcut”, okuyamıyorsa “mevcut değil” yazar.
- `$#` ile argüman sayısının 3’ten büyük olup olmadığına bakar. Büyükse kaç adet parametre girildiğini, değilse parametre sayısının yeterli olmadığını yazar.
- Üçüncü ve dördüncü argümanların aynı olup olmadığını karşılaştırır ve “aynı” ya da “farklı” yazar.
- Son olarak birinci argümandaki dosyanın `-r` ile okunabilir, `-w` ile yazılabilir olup olmadığını kontrol eder.

Değişkenlere ve argümanlara `$` ile erişilir: `$1` betiğe verilen birinci argüman, `$#` argümanların sayısıdır (number of arguments).

Anlatıcı `cat`’in bu kullanımını şöyle açıkladı: `cat` normalde dosyanın içeriğini göstermek için kullanılır, ama çalıştırıldığında bir sonuç döndürür ve bu sonuç dosyanın var olup olmadığıyla ilgilidir.

> **Ders sonrası not:** Derste `if cat $1` için “cat’in döndürdüğü birinci parametre dosyanın var olup olmadığını gösterir” dendi. Doğrusu: kabukta her komut bir çıkış durumu (exit status) döndürür. `cat` dosyayı okuyabilirse 0, okuyamazsa 0’dan farklı bir değer döner. `if` 0’ı doğru kabul eder. `$1` ise `cat`’in değil, betiğin birinci argümanıdır. Dosyanın varlığını kontrol etmenin doğrudan yolu `if [ -e "$1" ]` ya da `if [ -f "$1" ]` yazmaktır.

Anlatıcı betiği ilk denemede çalıştıramadı ve slayta sonra dönmeyi önerdi. Bu hata ayıklama dersin sonunda yapıldı (Bölüm 11).

9. Döngüler: `for`, `while`, `until` ve `let`

9.1 `for` Döngüsünün Biçimleri

Döngülerde de blok anahtar kelimelerle belirlenir: gövde `do` ile başlar, `done` ile biter. Aralık (range) vermek için birkaç seçenek vardır:

- Aralığın bütün elemanlarını küme gibi tek tek yazmak: `for i in 1 2 3 4 5`
- Bir aralık belirtmek: `for i in {1..5}`
- C’den alışık olduğumuz biçim: başlangıç ifadesi, durma koşulu ve artırma ifadesi. `for ((i=1; i<=5; i++))`. Anlatıcı bunu döngünün ihtiyaç duyduğu her şeyin parametrik olarak verildiği, matematiksel fonksiyon gibi bir gösterim olarak tanımladı.
- Adım vererek aralık belirtmek: `for i in {1..10..2}` birden ona kadar ikişer ikişer artar.
- Bir komuttan dönen listeyi dolaşmak: `for i in $(ls)` ile `ls`’nin verdiği her ad sırayla `i`’ye atanır.
- Yol verip * kullanmak: `for i in ~/*` yine bir liste döndürür ve dolaşılabilir.

Anlatıcı örneği çalıştırdı:

- İlk üç döngü aynı işi yaptı ve 1’den 5’e kadar yazdı. Bu beklenen sonuçtu.
- `{1..10..2}` döngüsü 1, 3, 5, 7, 9 yazdı.
- `ls` döngüsü bulunulan dizindeki dosyaları yazdı.

```
for i in 1 2 3 4 5; do echo "sayac $i"; done       # 1 2 3 4 5
for i in {1..5}; do echo "sayac $i"; done          # 1 2 3 4 5
for ((i=1; i<=5; i++)); do echo "sayac $i"; done   # 1 2 3 4 5
for i in {1..10..2}; do echo "sayac $i"; done      # 1 3 5 7 9
```

9.2 Ev Dizinindeki Öğelerin Tipini Yazdırma

Slaytın sağ tarafındaki örnek ~ ile ev dizinindeki bütün öğeleri tek tek dolaşır. Her öğenin bir dosya mı yoksa bir klasör mü olduğunu yazdırır:

```
for i in ~/*; do
  if [ -f $i ]; then echo "dosya : $i"
  elif [ -d $i ]; then echo "klasor : $i"
  else echo "bilemedim bu ne : $i"
```

```
  fi
done
```

Anlatıcı çıktıyı ev dizininin içeriğiyle karşılaştırdı. Aynı öğeler listelenmişti ve hangilerinin dosya, hangilerinin klasör olduğu yazılmıştı. Bu örnek, döngünün içinde koşul yapısının da kullanılabildiğini gösterir.

9.3 `while` Döngüsü

`for` dışında `while` ve `until` döngüleri de vardır. Bunlar da bir koşul ifadesiyle çalışır ve koşul yine köşeli parantez içinde yazılır.

```
i=3
while [ $i -gt 0 ]; do
  echo "sayac $i"
  i=`expr $i - 1`
done                        # sayac 3, sayac 2, sayac 1
```

`$i` bir değişkendir. `-gt` işleci "greater than", yani büyüktür demektir. `$i` sıfırdan büyük olduğu sürece döngü döner. Her turda `i` bir azaltılır. Azaltma için önceki hafta görülen `expr` ifadesi ters tırnak içinde kullanılır: `$i`'deki değerden 1 çıkarılıp sonuç tekrar `i`'ye atanır. Anlatıcıya göre bu, bildiğimiz "i sıfırdan büyükken i = i - 1" döngüsünün kabuktaki yazımıdır.

Çalıştırıldığında döngü sayacı geriye doğru, sıfırdan büyük kaldığı sürece azalta azalta yazdırdı.

9.4 `until` Döngüsü ve `let`

İkinci örnekte bir sayaç (counter) değişkeni vardır. `until` döngüsü koşul doğru olana kadar döner. Bu örnekte sayaç 3'ten küçük olana kadar her turda 1 azaltılır.

Burada `i=$((i-1))` ya da `expr` gibi bir yazım yerine `let` kullanılır:

```
let "sayac-=1"
```

Bu yazım, `sayac=$((sayac-1))` ile aynı işi yapar. Anlatıcıya göre `let` hem yazması daha kolay hem de göze daha iyi hitap eden bir biçimdir. C'deki alışık olduğumuz biçimde yazmak isteyenler `let`'i kullanabilir. Döngünün içinde sayaç `echo` ile ekrana yazdırılır.

Özetle `while` koşul doğru oldukça, `until` koşul doğru olana kadar döner. İkisi de `do ... done` bloğuyla yazılır.

> **Ek bilgi (derste anlatılmadı):** `let` yerine bash'te `(( ))` yazımı da kullanılabilir: `((sayac-=1))` ya da `((i--))`. `$(( ))` ise aritmetik ifadenin değerini döndürür ve atamalarda kullanılır, örneğin `i=$((i-1))`. Bu yazımlar `expr`'den farklı olarak `*` için kaçış karakteri gerektirmez.

10. `case` Yapısı

`case` yapısı C'deki `switch-case`'e karşılık gelir. Desenlerin sonundaki `*)` durumu, C'deki `default`'a denk gelir: önceki durumlardan hiçbirine uymayan her şey buraya yönlendirilir.

`if`'in `fi` ile kapanması gibi, `case` de kelimenin tersten yazılışı olan `esac` ile kapanır. Dikkat edilmesi gereken bir nokta, her durumun sonundaki iki noktalı virgüldür (`;;`). Her bir durumu (case statement) diğerinden ayırmak için `;;` kullanılır.

```
case $x in
  desen1) komutlar ;;
  desen2) komutlar ;;
  *) varsayılan komutlar ;;
esac
```

Anlatıcı örneği çalıştırdı. Betik kullanıcıdan `read` ile bir değer okudu. Anlatıcı sırayla "araç", "ev" ve rastgele bir değer girdi. Her girdi için ilgili durumun çıktısı göründü. Rastgele değer `*)` durumuna düştü.

> **Hocanın vurgusu:** `case` yapısında her durum `;;` ile biter; bu önemli bir ayırıcıdır. `case` ile yapılan her şey `if` ile de yapılabilir, ama `case` genelde göze daha iyi hitap eder.

Anlatıcı, kabuk fonksiyonlarını içeren 17 numaralı örneğin (`17_function.sh`) slaytlara konmadığını, bu içeriğin sonraki yıl ekleneceğini söyledi. Bu derste fonksiyonlar gösterilmedi.

11. Sorgu Betiğinin Hata Ayıklaması

Sunum bitince anlatıcı, ilk denemede çalışmayan sorgu betiğine geri döndü. Betiği adım adım inceledi. Bir dosya oluşturuluyor, içine `echo` ile bir metin yazılıyordu.

Bu sırada katılımcılardan biri argüman vermek gerektiğini söyledi. Anlatıcı da sorunun bu olduğunu kabul etti: betik çalıştırılırken argümanlar verilmemişti. Birinci argüman dosyanın adıdır. Kodun geri kalanında da üçüncü ve dördüncü argümanlar kullanılıyordu. Kodu okumadan, argüman vermeden çalıştırmaya çalışınca betik beklenen işi yapmaz.

> **Hocanın vurgusu:** Betiği çalıştırırken argümanları vermeyi unutmayın. Örnek ilk denemede argümanlar verilmediği için çalışmadı.

11.1 Denemeler

Anlatıcı önce `3 4 5` argümanlarını verdi ve “dört argüman var, biri dosyanın kendisi” dedi. Çıktı şöyleydi:

- “Böyle bir dosya ya da dizin yok” (No such file or directory) hatası.
- “3 isimli dosya mevcut değil.”
- “3 adet parametre yeterli değil.”

Sonra daha fazla argüman verdi: `4 3 2 1`. Bu sefer çıktıda “4 adet parametre girdiniz” yazdı. Üçüncü ve dördüncü argümanlar karşılaştırıldığında, girilen 2 ile 1 farklı olduğu için “farklı” yazdı.

Son olarak birinci argümanın yerine gerçek bir dosya adı yazdı: `dosya.txt`. Bu kez betik “dosya.txt yazılabilir bir dosyadır” çıktısını verdi. Yani ilk parametrede gelen dosyanın yazılabilir olduğu `-w` koşuluyla bulunmuştu.

> **Ders sonrası not:** Anlatıcı ilk denemede dört argüman verdiğini ve birinin dosya adı olduğunu söyledi, ama verilen argümanlar yalnızca `3 4 5` idi. Dosya adı verilmemişti. Bu yüzden `$1` = 3 oldu: `cat 3` hata verdi, “3 isimli dosya mevcut değil” yazıldı. `$#` = 3 olduğu için “3 > 3” yanlıştır ve “3 adet parametre yeterli değil” çıktısı geldi. İkinci denemede de (`4 3 2 1`) dosya adı yoktu: `$#` = 4, `$3` = 2, `$4` = 1 oldu. “4 adet parametre girdiniz” ve “2 ile 1 farklı” sonuçları bununla tutarlıdır. Bu çıktılar, dersteki anlatıma göre kurulan bir betik iskeletiyle scratchpad’de bash ile yeniden üretildi. Doğru çalıştırma, dosya adını birinci argüman olarak vermektir: `./13_queries.sh dosya.txt 4 3 2 1`.

12. Uygulamanın Genel Değerlendirmesi ve Sonraki Uygulamalar

Anlatıcı uygulamayı şöyle toparladı:

- Döngülerin sözdizimini (syntax) öğrenmek önemlidir.
- `if` ve `while` yapıları birbirine çok benzer. İkisinde de `$` ile erişilen değişkenler, `-gt`, `-lt` gibi işleçlerle neyden büyük ya da küçük oldukları belirtilerek karşılaştırılır.
- Döngülerde aralık vermenin farklı yolları vardır.
- Dosyanın tipi ve okunabilir ya da yazılabilir olup olmadığı, koşullarda başına `-` konan seçeneklerle sorgulanır.
- Alışık olunan biçimde yazmak isteyenler için `let` vardır.
- `case`’te `;;` ayırıcısına dikkat edilmelidir.
- `awk` ile tablo biçimli dosyalar filtrelenir, `tr` ile karakter ve string dönüşümleri yapılır, pipe ise bütün bunları birbirine bağlar.

Anlatıcıya göre gösterilen temel yapılar herhangi bir kabuk betiği yazmak için yeterlidir. Amacı, kabuğun sözdiziminin ve çalışma biçiminin diğer programlama dillerinden farklarını

göstermekti. Programlama dillerindeki genel yapıların hepsi gösterildi, yalnızca fonksiyonlar gösterilmedi.

> **Hocanın vurgusu:** Kabuk programlamada uzman olmanız gerekmiyor. Ama hayatınızın bir döneminde mutlaka kullanmanız gerekecek. “Ben veri bilimi çalışacağım, kabukla işim olmaz” diye düşünebilirsiniz, ama bir gün bir program kurmanız gerekecek ve bu komutlarla karşılaşacaksınız. Bu yüzden onları görünce kodun ne yaptığını anlayabilmeniz önemlidir.

Dönemin sonraki uygulamalarında sırasıyla süreçler arası iletişim (interprocess communication), ardından iş parçacıkları (thread) görülecek. Dönem sonunda da çekirdek modüllerinden (kernel module) bahsedilecek. Bu uygulamalarda verilenler doğrudan kullanılmayabilir ama bir başlangıç noktasıdır (starting point).

Bir soruya cevap olarak anlatıcı, uygulamalardan bir proje verilebileceğini söyledi. Problem verilecek, tasarım ise tamamen öğrencilere kalacak. Bu, algoritma analizi ve veri yapıları gibi derslerdeki projelere benzer: problem bellidir, çözümü öğrenci üretir. Anlatıcıya göre büyük ihtimalle bir çekirdek modülü istenecek, ancak bu henüz kesin değil.

Örnek Sorular

Soru 1 (derste sözlü, 00:33:27): `birdosya.txt` üzerinde `awk -F ';' 'int($2)>3'` çalıştırıldığında neden yalnızca tek bir satır yazılır?

> **Çözüm:** `-F ';'` her satırı noktalı virgüle göre böler. `$2` ikinci sütundur. İkinci sütunları 4, 3, 2, 1, 0 olan satırlar arasında 3’ten büyük olan yalnızca 4’tür. Bu yüzden yalnızca `ahmet;4;1` satırı yazılır.

Soru 2 (derste sözlü, 00:34:15): Aynı dosyada ayırıcı `;` yerine `,` verilirse filtre nasıl davranır?

> **Çözüm:** Dosyada virgül olmadığı için satır bölünemez. Bütün satır `$1`’e düşer, `$2` ve `$3` boş kalır. Derste değiştirilen `int($3)<=4` komutu bu durumda bütün satırları yazdı, çünkü boş alan `int()` ile 0 olur ve $0 \leq 4$ her satırda doğrudur. Bash ile denendiğinde `int($2)>3` ise hiçbir satır yazmaz (bu kısım derste denenmedi).

Soru 3 (derste sözlü, 00:53:09 ve 00:54:17): Sorgu betiği neden ilk denemelerde beklenen sonucu vermedi?

> **Çözüm:** Derste katılımcının önerisiyle bulunan neden, argümanların verilmemesiydi. Betik birinci argümanı dosya adı olarak, üçüncü ve dördüncü argümanları karşılaştırma için kullanır. `3 4 5` ile çalıştırıldığında dosya adı yoktur: `$1` = 3 olduğundan `cat 3` hata verir, `$#` = 3 olduğundan “3’ten büyük” koşulu sağlanmaz. Birinci argüman olarak `dosya.txt` verildiğinde dosya bulunur ve `-w` koşulu “yazılabilir” sonucunu verir.

Dersin Toparlaması

Bu uygulamada kabuk programlamanın ikinci yarısı tamamlandı. Ders, geçen hafta eksik kalan kullanıcı ve grup dosyalarıyla başladı. Ardından pipe ile bir komutun çıktısının başka bir komuta nasıl aktarıldığı görüldü. `grep` ile uzun listeler filtrelendi, `tr` ile karakterler dönüştürülüp silindi, `awk` ile ayırıcısı belli tablo biçimli dosyalar sütun bazında filtrelendi. `bc` ile terminalde yapılan hesapların betiğe de konabileceği gösterildi. Sonra programlamanın temel yapıları geldi: `if ... fi` koşulları ve `test` işleçleri, `for`, `while`, `until` döngüleri, `let` ile aritmetik ve `case ... esac` yapısı. Kabukta blokların süslü parantezle değil, anahtar kelime çiftleriyle belirlendiği vurgulandı.

Dersin sonunda, çalışmayan sorgu betiğinin sorununun eksik argümanlar olduğu bulundu. Sonraki uygulamalarda süreçler arası iletişim, iş parçacıkları ve çekirdek modülleri işlenecek. Kabuk programlama, bu uygulamalar ve verilebilecek bir proje için başlangıç noktasıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.